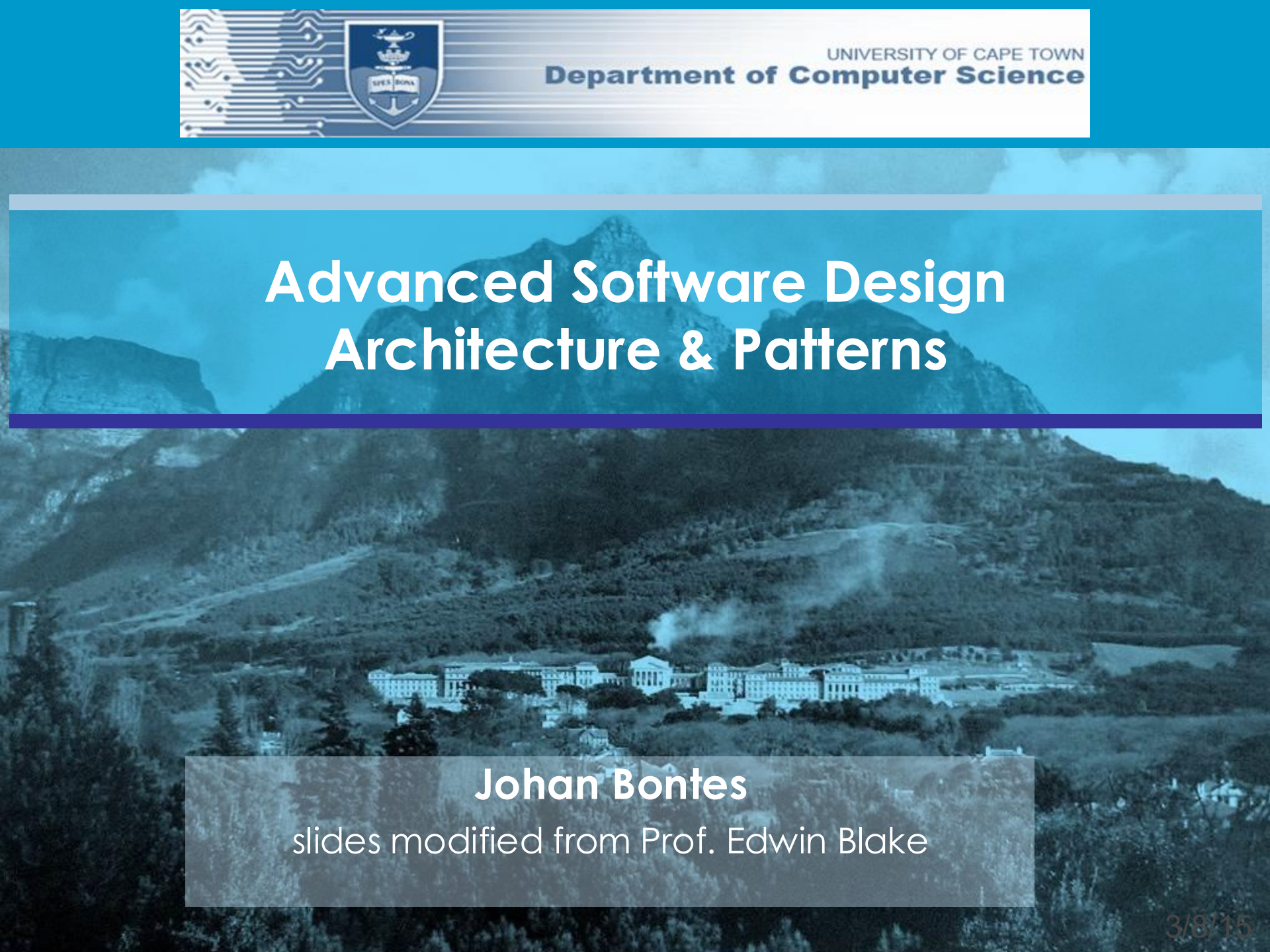
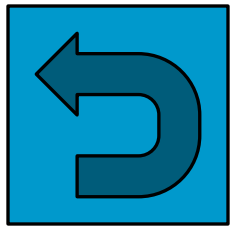


Advanced Software Design Architecture & Patterns

Johan Bontes

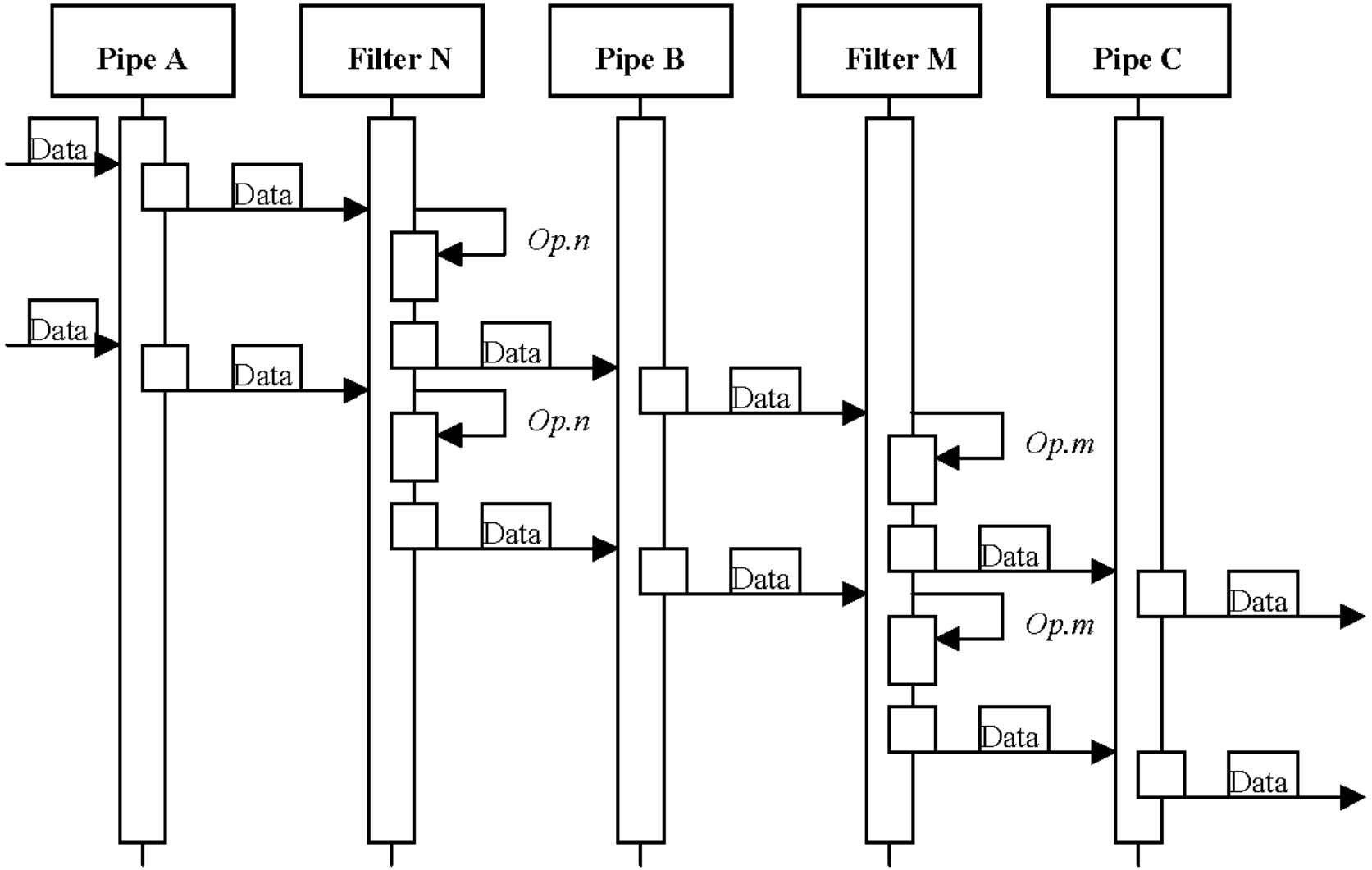
slides modified from Prof. Edwin Blake

Pipeline or Pipe and Filter arch.



- Suitable for apps that require a defined series of independent computations to be performed on ordered data.
  - Very useful if each computation can be done incrementally on the data
    - Sometimes computations can proceed in parallel.
- Decomposes a problem into a set of computations (filters), with operations (pipes) to stream data from one process to another.
  - filters interact only via pipes
  - “pure” filters have local processing and little state.
- Common in Unix
  - Unix “filters” often buffer all input before producing output
    - Can cause starvation if the data flow topology includes loops.

Parallel Pipe and Filter Diagram



CONCLUSION

Software Architecture

Introducing Patterns

Key Architecture Patterns

Model-View-Controller

Layered Architecture

Patterns and Concurrency

Conclusion

Realizing Non-functional Requirements

Performance

- Localize critical operations and minimize communications.
- Use large rather than fine-grain components. Allow for concurrency.

Security

- Use a layered architecture with critical assets in the inner layers.

Safety

- Localize safety-critical features in a small number of sub-systems.

Availability

- Include redundant components and mechanisms for fault tolerance.
- Allow replacement of components without stopping the system.

Maintainability

- Use fine-grain, replaceable components.
- Avoid global/shared data structures.

Purpose of Architecture: beforehand

- Software architecture can serve as a design plan for the negotiation of system requirements
- A means of structuring discussions with clients, developers, and managers.
- It is an essential tool for complexity management. It hides details and allows the designers to focus on the key system abstractions.

OUTLINE

Application Architectures

Data processing systems

Transaction processing systems

Information and resource management systems

Event processing systems

Language processing systems

Conclusion

Application Perspective

- So far had architectural perspectives on issues such as control, distribution and system structuring.
- Now an alternative approach: architectures from an application perspective
  - Two fundamental models of business systems
    - batch processing
    - transaction processing
  - Event processing systems
  - Structure of language processing systems

Use of application architectures

- As a starting point for architectural design.
- As a way of organising the work of the development team.
- As a means of assessing components for reuse.
- As a vocabulary for talking about application types.

Application types

Data processing applications

- Data driven applications that process data in batches without explicit user intervention during the processing.
  - Billing systems; Payroll systems

Transaction processing applications

- Data-centred applications that process user requests and update information in a system database.
  - E-commerce systems; Reservation systems

Event processing systems

- Applications where system actions depend on interpreting events from the system's environment.
  - Games; Word processors; Real-time systems

Language processing systems

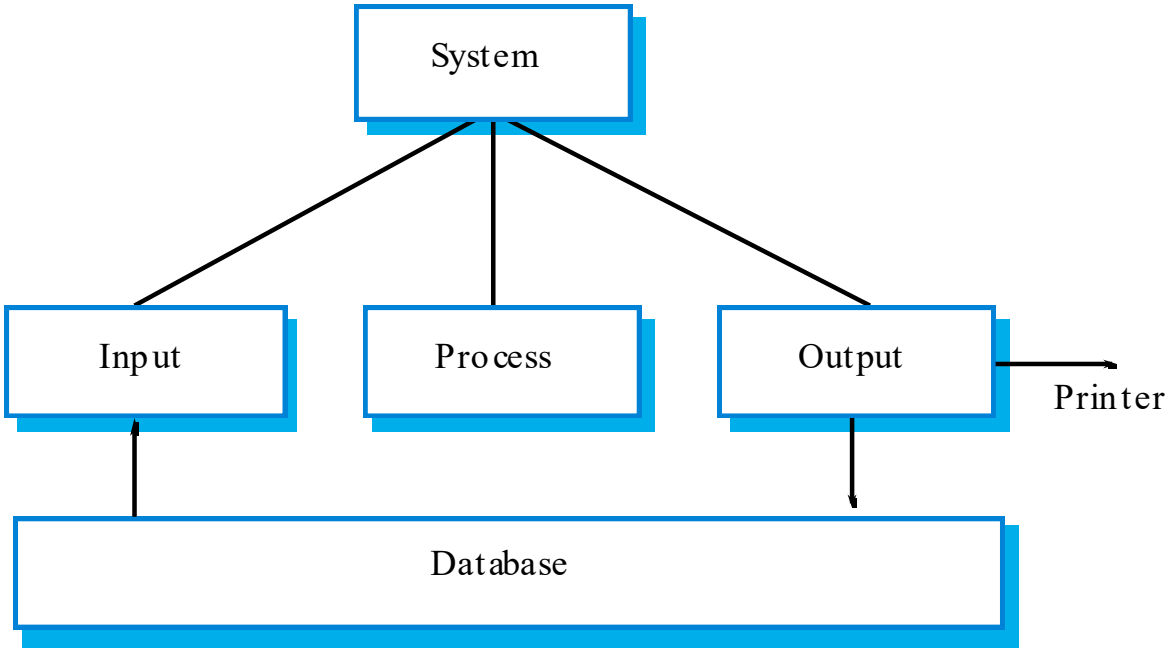
- Applications where the users' intentions are specified in a formal language that is processed and interpreted by the system.
  - Compilers; Command interpreters

Data Processing Applications - Input-process-output

input component reads data from a file or database, checks its validity and queues the valid data for processing.

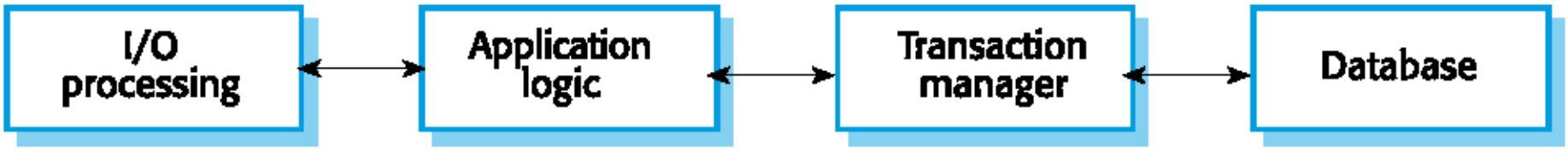
process component takes a transaction from the queue (input), performs computations and creates a new record with the results of the computation.

output component reads these records, formats them accordingly and writes them to the database or sends them to a printer.



Transaction processing systems

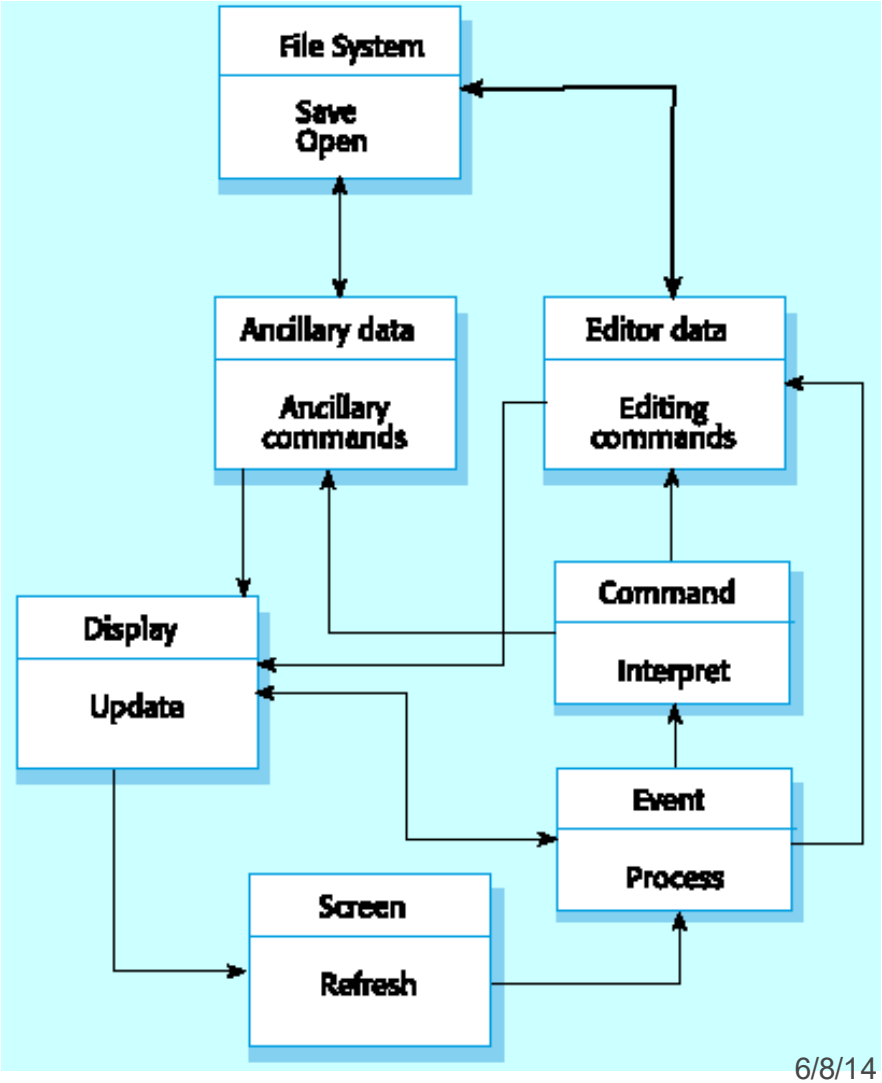
- Process user requests for information from a database or requests to update the database.
- From a user perspective a transaction is:
  - Any coherent sequence of operations that satisfies a goal;
  - For example - find the times of flights from London to Paris.
- Users make asynchronous requests for service which are then processed by a transaction manager.



Event Processing Systems - Editing system architecture

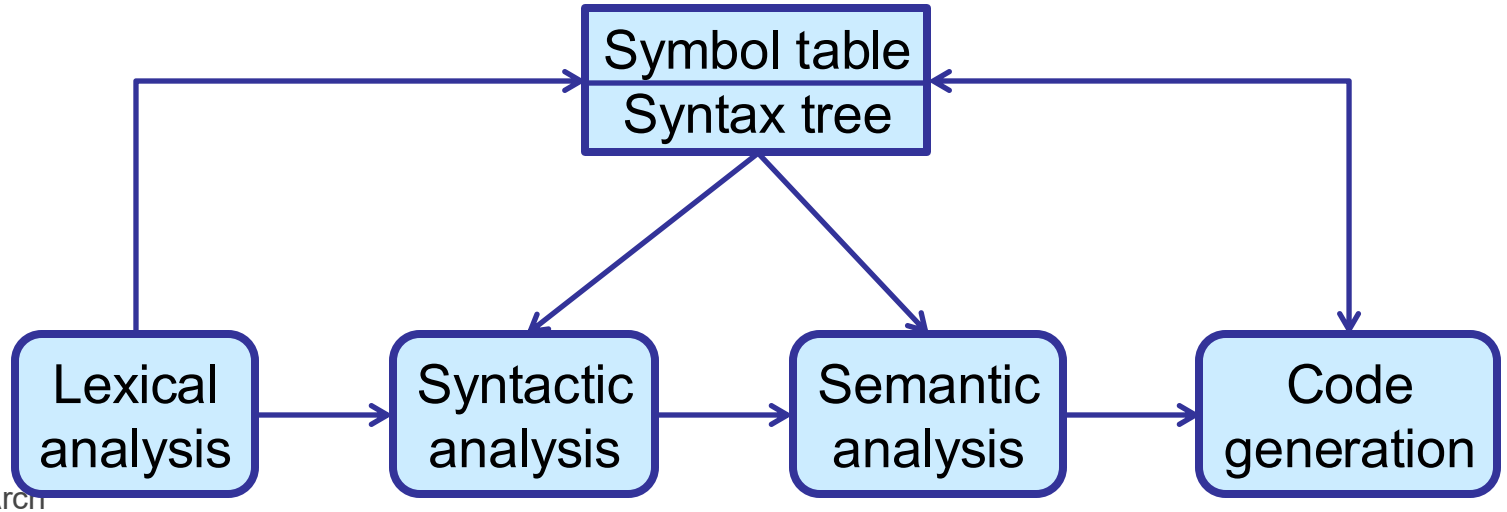
Editing systems are naturally object-oriented:

- **Screen**: monitors screen memory and detects events;
- **Event**: recognises events and passes them for processing;
- **Command**: executes a user command;
- **Editor data**: manages the editor data structure;
- **Ancillary data**: manages other data such as styles & preferences;
- **File system**: manages file I/O;
- **Display**: updates the screen display.



Compilers

- Lexical analyser
- Symbol table
- Syntax analyser
- Abstract Syntax Tree (AST)
- Semantic analyser
- Code generator



Conclusion

- Generic models of application architectures help us understand and compare applications.
- Application types are data processing systems, transaction processing systems, event processing systems and compilers.
  - or a combination
- Data processing systems operate in batch mode and have an input→process→output structure.
- Transaction processing systems allow information in a database to be remotely accessed and modified by multiple users.
- Event processing systems include editors and real-time systems.
  - In an editor, user interface events are detected and an in-store data structure is modified.
- Compilers translate source code into different language (e.g.: assembler) and may interpret the specified instructions.